

HYUNDAI-HR

도건우 이유경 이재우 홍준표

1차 프로젝트 : 현대백화점 임직원 전용 인사 관리 시스템 개발

인사, 근태, 휴가, 급여

목차

01 팀원 소개

02 시스템 전체 구조 (Architecture)

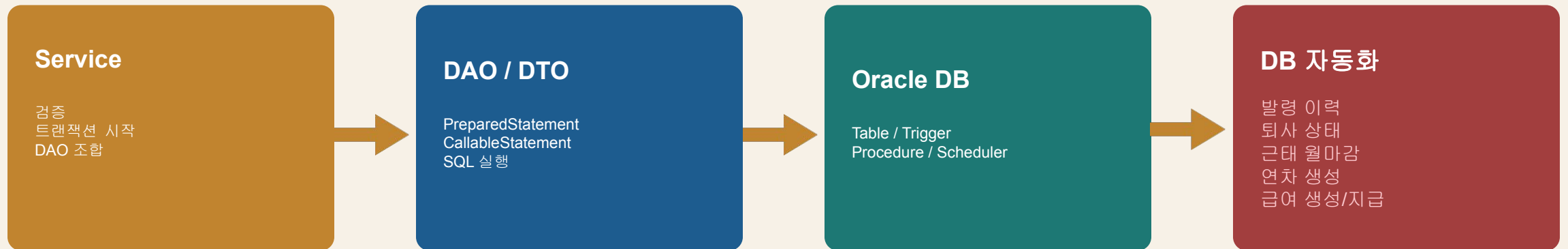
03 주요 기능 구현 (인사/근태/휴가/급여)

04 협업 전략 및 프로젝트 회고

05 프로젝트 시연 및 Q&A

직원 정보, 근태, 휴가, 급여를 Oracle DB로 관리

Service → DAO → Oracle DB로 구성



주요 기능 정리

인사

주요 기능

직원 등록·조회, 부서 등록/삭제, 조직도
조회, 인사 평가, 발령 이력

기술 포인트

트리거, 스케줄러, DTO 분리

근태

주요 기능

출퇴근 기록, 유연근무 신청, 정정 신청,
휴가 연동, 월 마감

기술 포인트

MERGE, 트랜잭션, 스케줄러

휴가

주요 기능

신규 입사자&기존 사원의 연차 자동
생성, 휴가 신청, 관리자 승인/반려, 연차
차감, 근태 반영

기술 포인트

Strategy, Factory, 트랜잭션, 스케줄러,
트리거, 프로시저

급여

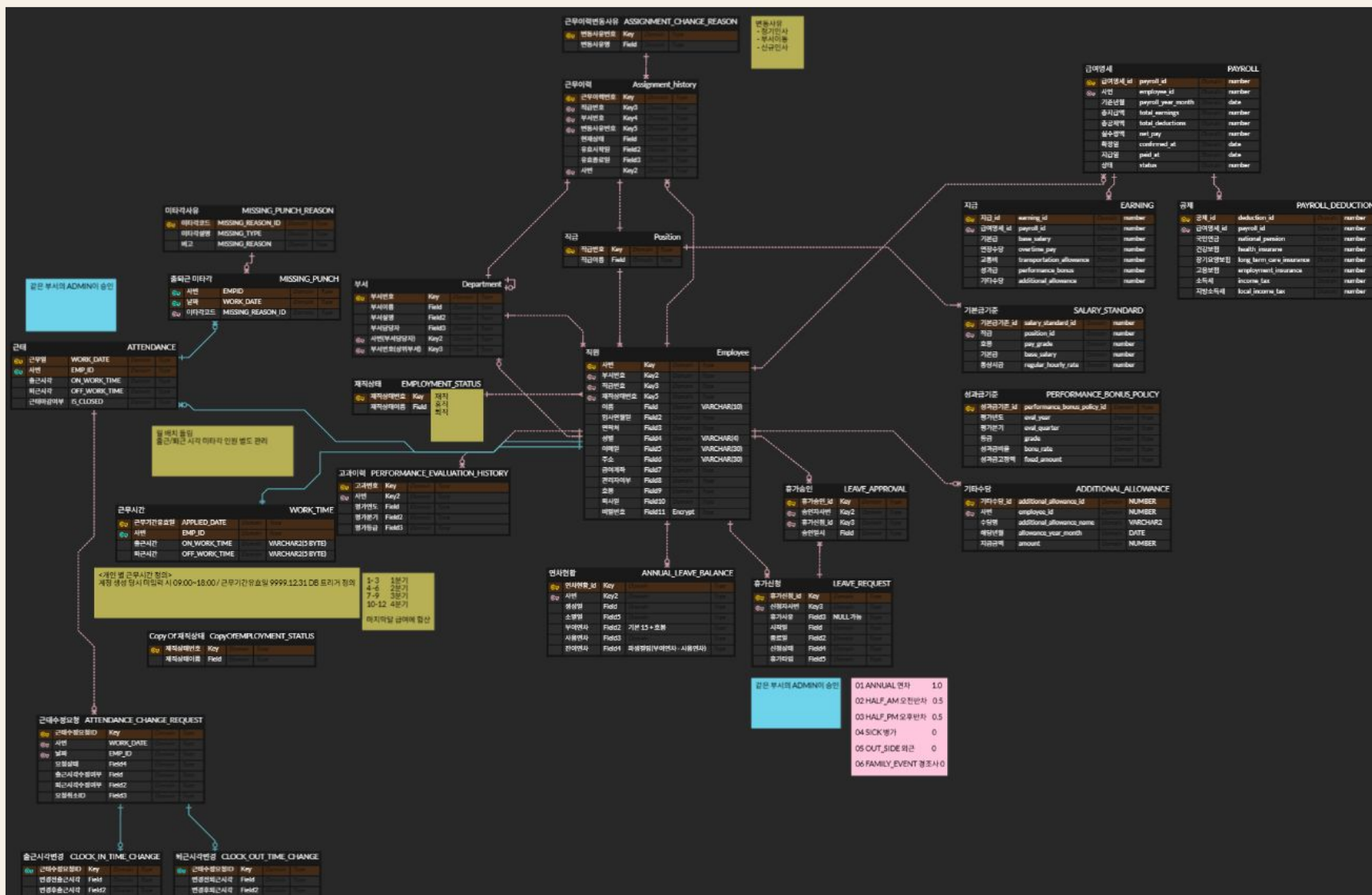
주요 기능

급여 생성, 목록/상세 조회, 확정/지급,
수정/삭제, 기타수당

기술 포인트

Procedure, Trigger, Transaction

ERD



공통 상태값은 휴가 결재와 급여 상태 전이에 사용

상태가 바뀌는 기능은 서비스 검증과 DB 제약 조건을 함께 사용했습니다.

휴가 / 근태 정정

PENDING
APPROVED
REJECTED
CANCELED

급여

CALCULATED
CONFIRMED
PAID

휴가 유형

ANNUAL
HALF_AM / HALF_PM
OUT_SIDE
SICK / FAMILY_EVENT

급여 상태 전이는 **CALCULATED** → **CONFIRMED** → **PAID**로 고정하고, 제대로 된 전이가 아니면 **rollback**합니다.

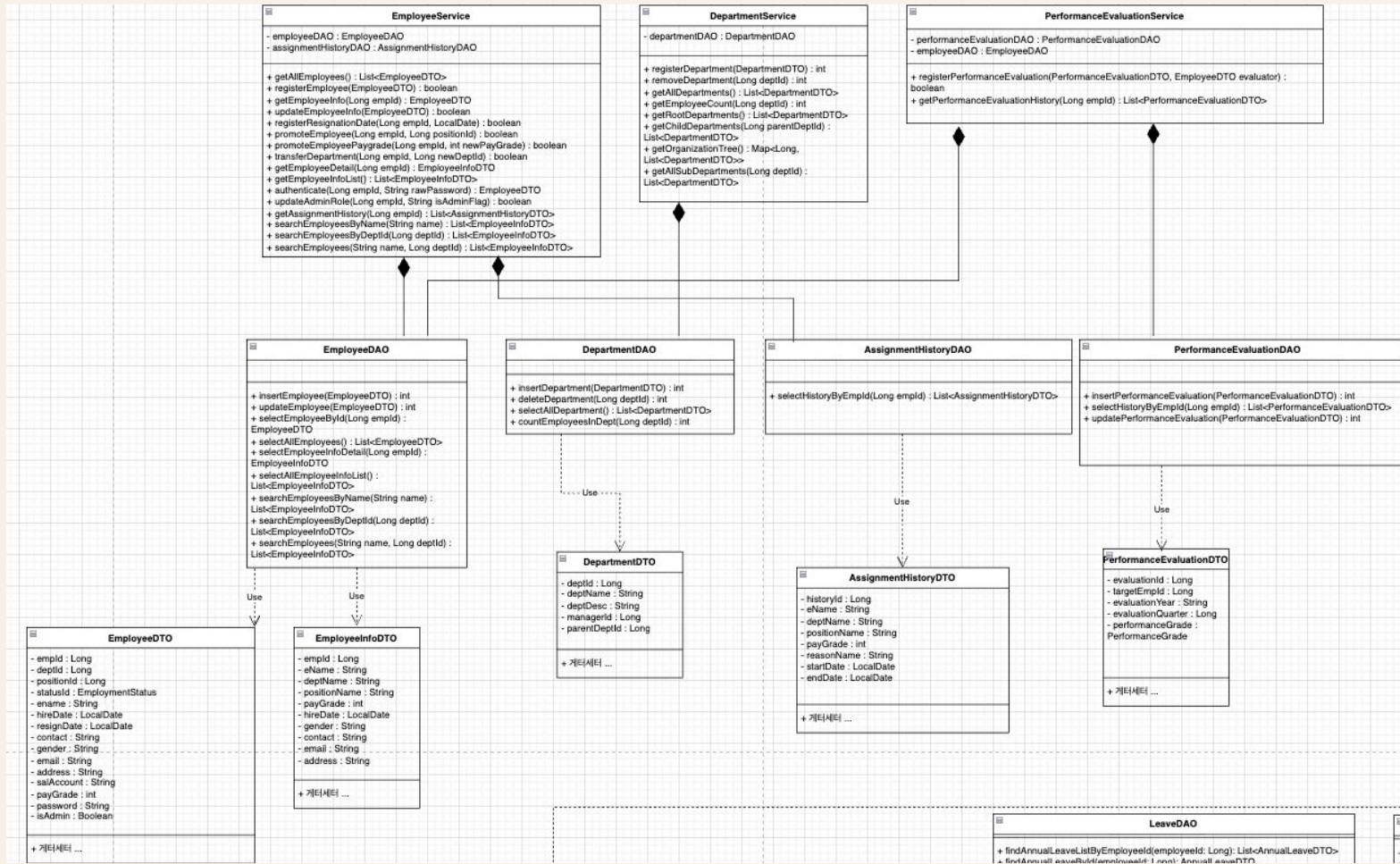
01

인사

직원 관리, 조직 관리, 인사 평가, 발령 이력

- **직원 관리**
신규 직원 등록, 직원 조회, 조건 검색 기능
- **조직 관리**
부서 등록/삭제, 조직도 조회
- **발령 이력**
승진, 부서 이동, 호봉 상승에 따른 이력 부여/조회

인사 관련 클래스 다이어그램



EmployeeDTO만 사용 시 외래키 숫자가 노출되는 문제

사용자에게 친숙한 정보를 제공하기 위해 맵핑용 DTO와 UI 출력 전용 DTO를 분리 설계하였습니다.

개선 전 (EmployeeDTO)

사번: 1001
이름: 홍길동
부서: 10
직급: 2



개선 후 (EmployeeInfoDTO)

사번: 1001
이름: 홍길동
부서: 개발팀
직급: 대리

DAO 단에서 **JOIN**을 통해 이름을 가져와 담아줌으로써 **직관적인 화면**을 제공합니다.

DAO JOIN 결과를 EmployeeInfoDTO에 담아 부서명·직급명 조회

DB 테이블과 1:1 매핑된 EmployeeDTO와 화면 출력용 EmployeeInfoDTO를 분리했습니다.

EmployeeDAO.selectEmployeeInfoDetail()

```
SELECT e.EMP_ID,  
e.NAME,  
d.DEPT_NAME,  
p.POSITION_NAME,  
e.PAY_GRADE  
FROM EMPLOYEE e  
LEFT JOIN DEPARTMENT d  
ON e.DEPT_ID = d.DEPT_ID  
LEFT JOIN POSITION p  
ON e.POSITION_ID = p.POSITION_ID  
WHERE e.EMP_ID = ?;  
  
info.setDeptName(rs.getString("DEPT_NAME"));  
info.setPositionName(rs.getString("POSITION_NAME"));
```

처리 기준

문제

부서코드: 10, 직급: 2처럼 외래키 숫자가 노출

해결

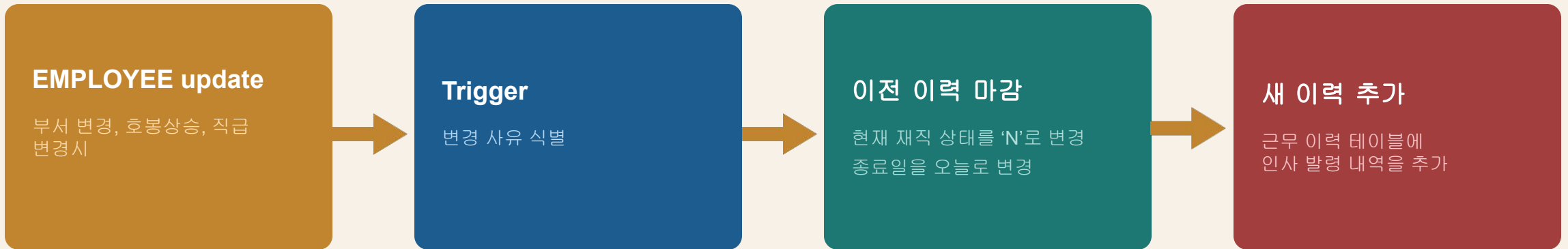
DAO 단에서 JOIN을 통해 이름을 가져와 담아줌

결과

부서: 개발팀, 직급: 대리

부서·직급·호봉 변경 시 트리거가 발령 이력을 자동 생성

승진, 호봉 상승, 부서 이동으로 인한 인사 정보 변경 시 발령 이력을 DB에서 남깁니다.



사용 이유

직원 테이블만 수정해도 발령 이력이 DB에서 일관되게 남습니다.

변경 사유 식별 후 현재 이력 마감

직급 변경은 승진, 부서 변경은 부서 이동, 호봉 변경은 호봉 상승으로 처리합니다.

trg_employee_assignment.sql

```
AFTER UPDATE OF dept_id, position_id, pay_grade
ON employee FOR EACH ROW

IF :OLD.position_id != :NEW.position_id THEN
v_reason_id := 2; -- 승진
ELSIF :OLD.dept_id != :NEW.dept_id THEN
v_reason_id := 3; -- 부서 이동
ELSIF :OLD.pay_grade != :NEW.pay_grade THEN
v_reason_id := 6; -- 호봉 상승
END IF;

UPDATE assignment_history
SET is_current = 'N',
end_date = TRUNC(SYSDATE)
WHERE emp_id = :NEW.emp_id
AND is_current = 'Y';
```

처리 기준 (Reason ID)

승진

position_id 변경 → ID: 2

부서 이동

dept_id 변경 → ID: 3

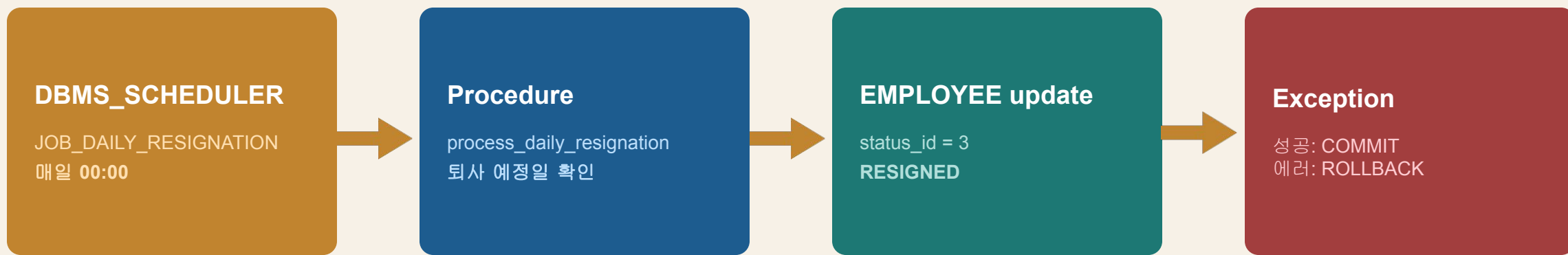
호봉 상승

pay_grade 변경 → ID: 6

이력ID	이름	부서	직급	호봉	사유	시작일	종료일
155	강하운	울산공장	사원	30	승진	2026-05-17	
154	강하운	울산공장	매니저	30	부서 이동	2026-05-17	2026-05-17
152	강하운	인사팀	매니저	30	호봉상승	2026-05-17	2026-05-17
151	강하운	인사팀	매니저	29	호봉상승	2026-05-17	2026-05-17

퇴사 예정일 도래 직원은 매일 자정에 퇴사 상태로 변경

대량의 퇴사 대상을 자바 단으로 끌어오지 않고 DB 내부에서 일괄 처리합니다.



효과

백엔드 서버의 네트워크 I/O 및 메모리 부하를 제거하여 시스템 안정성을 향상합니다.

Oracle Scheduler를 이용한 퇴사자 자동처리

퇴사 예정일이 도래한 직원의 상태를 갱신하는 작업을 매일 자정에 동작하는 Oracle Scheduler와 프로시저로 구현했습니다.

trg_employee_assignment.sql

```
CREATE OR REPLACE PROCEDURE process_daily_resignation IS
BEGIN
  UPDATE employee SET status_id = 3
  WHERE resign_date IS NOT NULL
  AND TRUNC(resign_date) <= TRUNC(SYSDATE);

  COMMIT;
  EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
END;

BEGIN
  DBMS_SCHEDULER.CREATE_JOB (
    job_name => 'JOB_DAILY_RESIGNATION',
    ...
    repeat_interval => 'FREQ=DAILY;BYHOUR=0'
    ...
  );
END;
```

구현 프로세스

1. 매일 자정 스케줄러 실행

DBMS_SCHEDULER가 정해진 시간에 JOB을 호출합니다.

2. 퇴사 대상자 식별

퇴사 예정일(resign_date)이 현재 날짜보다 이전인 데이터를 필터링합니다.

3. 상태값 업데이트

status_id를 '3'(RESIGNED)으로 변경

4. 트랜잭션 관리

성공 시 COMMIT, 예외 발생 시 ROLLBACK을 통해 데이터 관리

근태

출퇴근 기록, 유연근무, 정정 신청, 휴가 연동,
월 마감

- **출퇴근 기록**

출근과 퇴근은 같은 attendance row를 공유

- **유연근무**

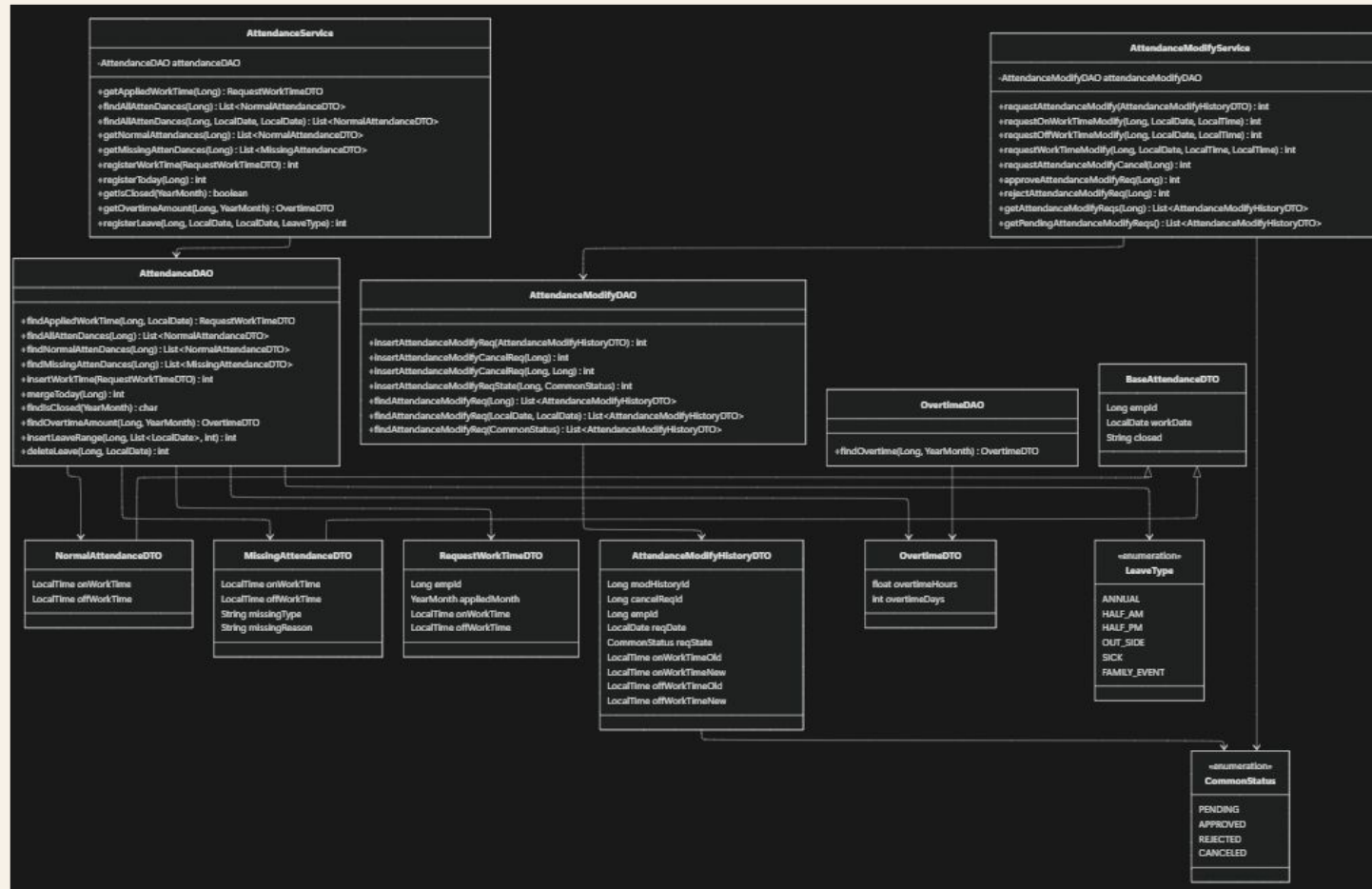
별도 변경 신청이 없을 경우 계속 유지

- **월 마감**

마감된 근태 정보는 더 이상 수정 불가능

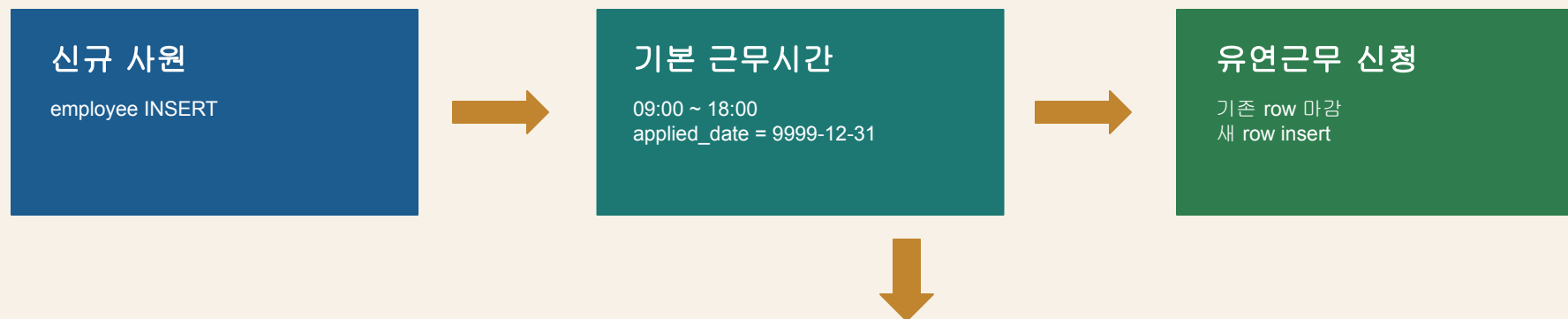
근태 데이터는 급여 계산과 연차 차감의 기준 데이터입니다.

근태 관련 클래스 다이어그램



work_time : 근태 판정의 기준 테이블

초과근무 수당 계산, 지각/조퇴 판별, 월 근태 마감의 기준 시간입니다.



이력형 구조

과거 근태는 당시 기준 근무시간으로 판별하고, 현재 또는 미래 근무시간만 새 기준으로 판단합니다.

	EMP_ID	ON_WORK_TIME	OFF_WORK_TIME	APPLIED_DATE
1	1010	09:00	18:00	2026-05-31 00:00:00
2	1010	10:00	19:00	2026-07-31 00:00:00
3	1010	09:00	18:00	9999-12-31 00:00:00

사원 유연근무 데이터 정합성 향상 : Trigger

현재 재직 중인 모든 인원은 work_time 테이블에 본인의 근무시간 정보가 반드시 존재해야 한다.

work_time은 단순 설정값이 아니라 초과근무 수당 계산, 지각/조퇴 판별, 월 근태 마감의 기준 시간이 되는 핵심 테이블이다.

사원 정보가 생성될 때 Java 로직에서 work_time을 함께 insert할 수도 있지만, 해당 로직이 누락되면 직원은 존재하는데 근무시간 기준이 없는 불완전한 데이터가 생길 수 있다.

```
create or replace trigger trg_emp_insert_work_time
after insert on employee
for each row
begin
    insert into work_time ( emp_id, applied_date, on_work_time, off_work_time )
    values( :NEW.emp_id, DATE '9999-12-31', '09:00', '18:00' );
end;
```

	EMP_ID	ON_WORK_TIME	OFF_WORK_TIME	APPLIED_DATE
1	1010	09:00	18:00	2026-05-31 00:00:00
2	1010	10:00	19:00	2026-07-31 00:00:00
3	1010	09:00	18:00	9999-12-31 00:00:00

유연근무 신청 : Service 검증 후 update와 insert를 하나의 트랜잭션으로 처리

둘 중 하나만 실행되면 관련 기능이 모두 적용되지 않습니다.

검증	조건	실패 메시지	처리 위치
퇴근 시간	퇴근시간은 출근시간보다 늦어야 한다	퇴근시간이 출근시간보다 앞설 수 없습니다	AttendanceService.registerWorkTime
근무 시간	출근~퇴근 간격이 최소 9시간 이상	근무시간 최소는 8시간입니다	AttendanceService.registerWorkTime
적용 월	다음달 이후만 가능	유연근무 신청은 다음달 이후로 신청할 수 있습니다	AttendanceService.registerWorkTime
트랜잭션	기존 row 마감 + 새 row insert	rollback	AttendanceDAO.insertWorkTime

출근과 퇴근 : attendance row 공유

해당 날짜 row가 없으면 INSERT, 이미 row가 있으면 퇴근 시간만 UPDATE합니다.

AttendanceDAO.mergeToday()

```
MERGE INTO attendance a
USING (
  SELECT ? AS emp_id,
         TRUNC(SYSDATE) AS work_date,
         CAST(SYSTIMESTAMP AS TIMESTAMP) AS now_time
  FROM dual
) src
ON (a.emp_id = src.emp_id
    AND a.work_date = src.work_date)

WHEN MATCHED THEN
  UPDATE SET a.off_work_time = src.now_time
  WHERE a.off_work_time IS NULL

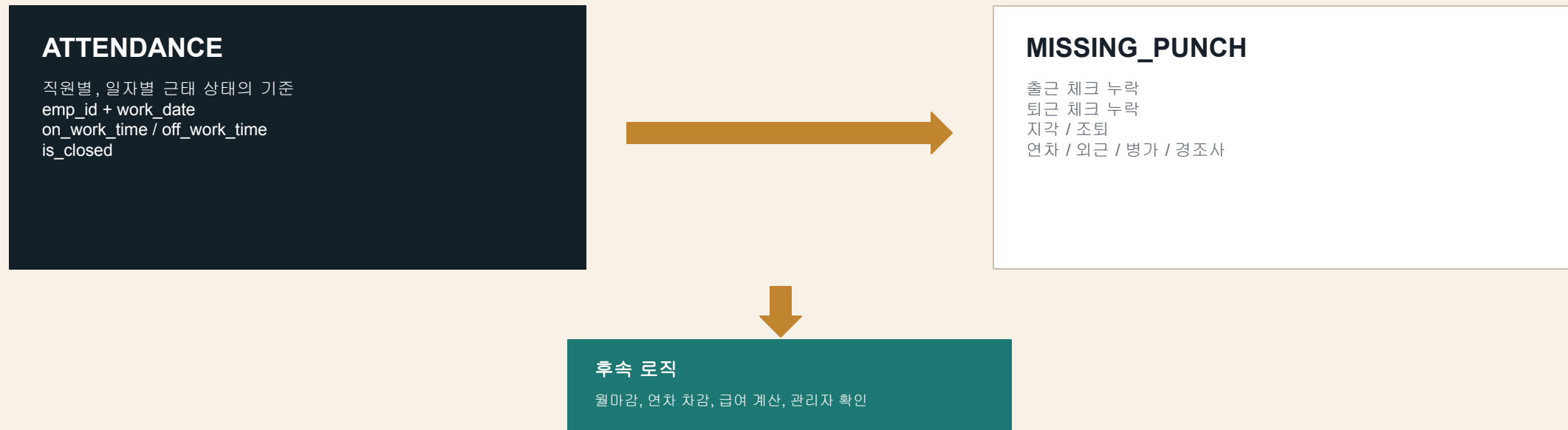
WHEN NOT MATCHED THEN
  INSERT (emp_id, work_date, on_work_time, is_closed)
  VALUES (src.emp_id, src.work_date, src.now_time, 'N');
```

처리 기준

- **판단**
SELECT 후 Java에서 분기하는 방식보다 DB의 MERGE가 적합
- **중복 방지**
emp_id와 work_date 기준으로 중복 row 생성 방지
- **로직 단순화**
출근 버튼과 퇴근 버튼을 하나의 SQL 흐름으로 처리

attendance는 원장, missing_punch는 예외 사유

정상 출퇴근 기록과 예외 근태 기록의 관리 목적이 다르기 때문에 분리했습니다.

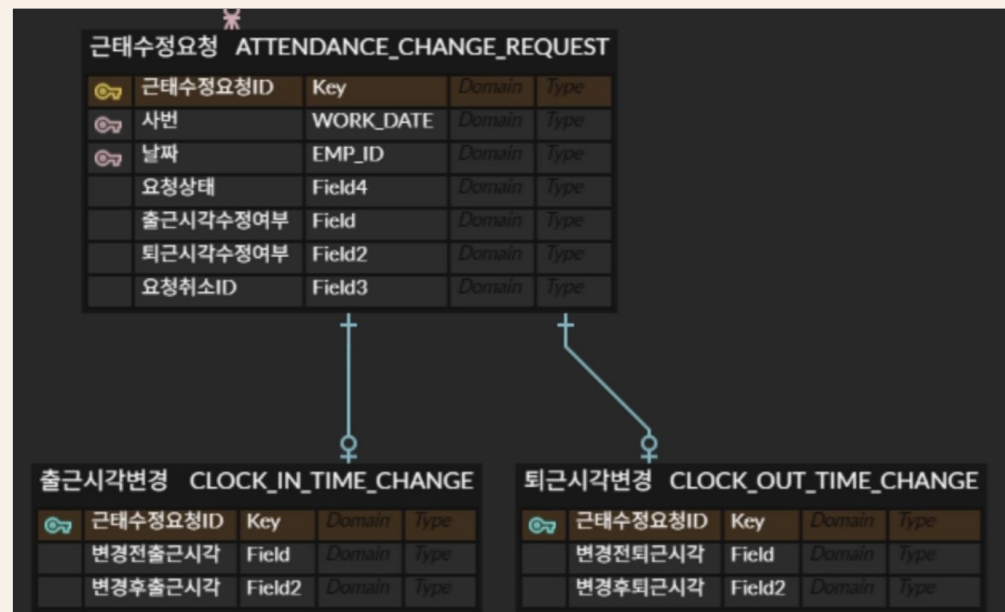


예외 사유가 늘어나는 경우에도 **attendance** 테이블에 컬럼을 계속 추가하지 않아도 됩니다.
실제 근태 정보를 다루는 테이블과 예외 근태 정보를 다루는 테이블을 분리함으로써, 역할 분리를 실현했습니다

근태 정정 신청 : 공통 요청 정보, 실제 변경 대상 데이터 분리 설계

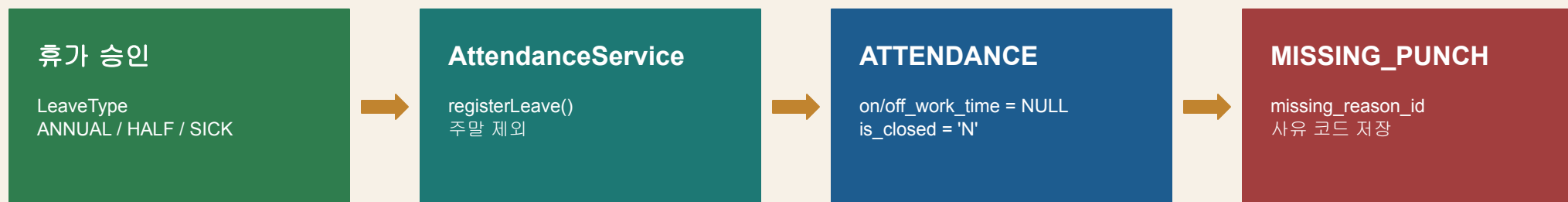
이렇게 분리한 이유는 출근만 변경하는 요청, 퇴근만 변경하는 요청, 출근/퇴근을 모두 변경하는 요청을 모두 표현하기 위해서다.

하나의 테이블에 모든 변경 컬럼을 넣으면 출근만 변경하는 요청에서는 퇴근 관련 컬럼이 계속 NULL로 남고, 요청 유형별 의미도 불명확해진다.



휴가 신청 건은 근태 정보로 반영

휴가, 외근, 병가, 경조사는 출퇴근 시간이 없는 근무일로 기록됩니다.



트랜잭션

두 테이블 insert는 하나의 트랜잭션으로 처리하고, 중복 키가 발생(해당 일자에 휴가 정보가 존재)하면 **rollback**합니다.

87	1040	2026-06-01 00:00:00	(null)	(null)	N
88	1040	2026-06-03 00:00:00	(null)	(null)	N
89	1040	2026-06-08 00:00:00	(null)	(null)	N
90	1040	2026-06-09 00:00:00	(null)	(null)	N
91	1040	2026-06-10 00:00:00	(null)	(null)	N

월 근태 마감은 매월 정해진 시점에 반복 수행되는 배치성 작업

전 직원의 한 달치 **attendance** 데이터를 대상으로 누락, 지각, 조퇴를 판별합니다.
근태 마감 이후엔 더이상 정정 신청이 불가능합니다



사용 이유

급여 계산과 연차 차감은 마감된 근태 데이터를 기준으로 수행되어야하기 때문입니다.

월 근태 마감은 매월 정해진 시점에 반복 수행되는 배치성 작업

CLOSE_MONTHLY_ATTENDANCE

```

MERGE INTO missing_punch mp
USING (
  --대상 근태 정보들 불러오기
  WITH target_attendance AS (
    SELECT
      a.emp_id,
      a.work_date,
      a.on_work_time,
      a.off_work_time
    FROM attendance a
    WHERE a.is_closed = 'N'
      AND a.work_date >= ADD_MONTHS(TRUNC(SYSDATE, 'MM'), -1)
      AND a.work_date < TRUNC(SYSDATE, 'MM')
  ),
  --대상 근태정보들에 적용되는 유연근무시간 연결
  work_time_matched AS (
    SELECT
      ta.emp_id,
      ta.work_date,
      ta.on_work_time,
      ta.off_work_time,
      w.on_work_time AS expected_on_text,
      w.off_work_time AS expected_off_text,
      ROW_NUMBER() OVER (
        PARTITION BY ta.emp_id, ta.work_date
        ORDER BY w.applied_date
      ) AS rn
    FROM target_attendance ta
    LEFT JOIN work_time w
      ON w.emp_id = ta.emp_id
      AND w.applied_date >= ta.work_date
  ),

```

월 근태 마감은 매월 정해진 시점에 반복 수행되는 배치성 작업

CLOSE_MONTHLY_ATTENDANCE

```
judged AS (  
  SELECT  
    emp_id,  
    work_date,  
    CASE  
      WHEN on_work_time IS NULL  
        AND off_work_time IS NOT NULL  
        THEN 1 -- 출근 체크 누락  
  
      WHEN on_work_time IS NOT NULL  
        AND off_work_time IS NULL  
        THEN 3 -- 퇴근 체크 누락  
  
      WHEN on_work_time IS NULL  
        AND off_work_time IS NULL  
        THEN 1 -- 둘 다 없으면 출근 체크 누락으로 대표 처리  
  
      WHEN expected_on_text IS NOT NULL  
        AND on_work_time >  
          CAST(work_date AS TIMESTAMP)  
          + NUMTODSINTERVAL(  
            TO_NUMBER(SUBSTR(expected_on_text, 1, 2)) * 60  
            + TO_NUMBER(SUBSTR(expected_on_text, 4, 2)),  
            'MINUTE'  
          )  
        THEN 4 -- 지각  
  
      WHEN expected_off_text IS NOT NULL  
        AND off_work_time <  
          CAST(work_date AS TIMESTAMP)  
          + NUMTODSINTERVAL(  
            TO_NUMBER(SUBSTR(expected_off_text, 1, 2)) * 60  
            + TO_NUMBER(SUBSTR(expected_off_text, 4, 2)),  
            'MINUTE'  
          )  
        THEN 6 -- 조퇴  
  
      ELSE NULL  
    END AS missing_reason_id  
  FROM work_time_matched  
  WHERE rn = 1  
)
```

월 근태 마감은 매월 정해진 시점에 반복 수행되는 배치성 작업

CLOSE_MONTHLY_ATTENDANCE

-src : 판정된 근태 종류

```
SELECT
    emp_id,
    work_date,
    missing_reason_id
FROM judged
WHERE missing_reason_id IS NOT NULL
) src
ON (
    mp.emp_id = src.emp_id
    AND mp.work_date = src.work_date
)
WHEN NOT MATCHED THEN
    INSERT (
        mp.emp_id,
        mp.work_date,
        mp.missing_reason_id
    )
    VALUES (
        src.emp_id,
        src.work_date,
        src.missing_reason_id
    );
```

+ 출퇴근 체크누락 / 연차 / 조퇴 -> 연차 차감
+ 직전 달 마감

휴가

휴가 정책, 휴가 신청, 관리자 승인, 연차 차감

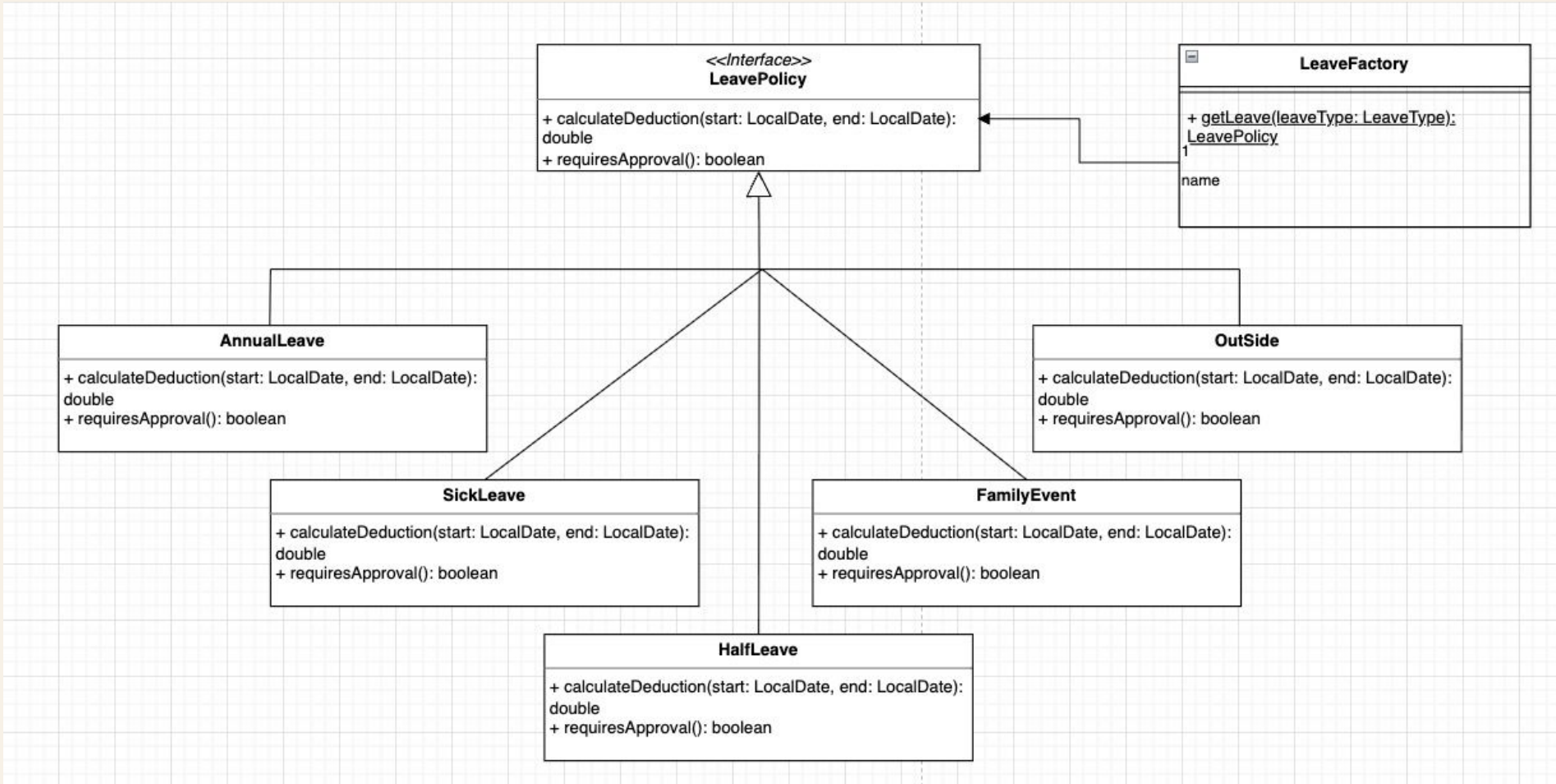
- **휴가 유형에 따라 전략 패턴 적용**
휴가 유형별 차감 규칙의 다양성을 수용하기 위해 상위 인터페이스 기반 전략 패턴 도입
가져온 구현체 내부의 휴가 여사 처리의 처리하는 비즈니스 규칙 명확화
- **각 휴가 구현체를 반환하는 심플 팩토리 패턴 적용**
객체 생성 책임을 LeaveFactory로 분리하여 서비스 계층과의 결합도 감소
정책 클래스를 몰라도 인터페이스 의존 원칙에 의해 동작 가능한 구조 구현
- **트랜잭션**
관리자의 휴가 승인 시 휴가 유형에 따라 연차 차감 프로세스가 일어나기 때문에 결재 상태 변경과 연차 차감은 원자적으로 실행
- **트리거, 프로시저, 스케줄러 기반의 DB 중심 시스템 구축**
애플리케이션 연산 부하 최소화 및 인위적 개입 없는 자동화
- **사용자 정의 예외 처리**
단순 예외 처리를 넘어 코드 레벨에서 행정 규칙과 의미를 명확히 규정

휴가 유형별 차감 규칙을 서비스 로직에게 위임하지 않기 위해 정책 클래스로 분리.
사용자 정의 예외를 커스텀하고 신규 사원 입사 및 연차 차감 반영 로직을 DB에서 자동화

휴가 유형별 차감 일수와 승인 필요 여부

휴가 유형	정책 클래스	차감 일수 계산	승인 필요 여부
연차	AnnualLeavePolicy	평일 기준 (1.0, 2.0...) / 주말 포함 신청 시 주말 제외 로직 발동	Yes
반차	HalfLeavePolicy	0.5 고정 / 시작일과 종료일이 같아야 함	Yes
병가	SickLeavePolicy	0.0	Yes
경조사	FamilyEventPolicy	0.0	Yes
외근	DutyLeavePolicy	0.0	No

전략 패턴 기반의 휴가 정책 추상화 : 클래스 다이어그램



LeaveType을 정책 객체로 변환

객체 생성 책임을 LeaveFactory로 격리.

→ 새로운 휴가 정책이 추가되더라도 가장 중요한 비즈니스 로직인 LeaveService는 수정하지 않도록 OCP 원칙 준수

LeaveFactory.getLeave()

```
public static LeavePolicy getLeave(LeaveType leaveType) {  
    switch (leaveType) {  
        case ANNUAL:  
            return new AnnualLeave();  
        case HALF_AM:  
        case HALF_PM:  
            return new HalfLeave();  
        case OUT_SIDE:  
            return new OutSide();  
        case SICK:  
            return new SickLeave();  
        case FAMILY_EVENT:  
            return new FamilyEvent();  
        default:  
            throw new IllegalArgumentException(...);  
    }  
}
```

처리 기준

- **AnnualLeave**

주말 제외 후 평일 근무일 카운트

- **HalfLeave**

무조건 0.5 반환

- **Sick / Family / OutSide**

연차 차감 0.0 반환

서비스 계층은 구체 클래스가 아니라 LeavePolicy만 사용

새로운 휴가 정책이 추가될 때마다 핵심 서비스 코드를 뜯어고치지 않도록 분리



서비스 계층은 내가 지금 사용하는 정책이 구체적으로 어떤 클래스인지, 어떤 휴가 정책을 지키는지 고려하지 않도록 구현

사용자 정의 예외를 통한 비즈니스 규약 정의

```
package global.exception;

public class BusinessException extends RuntimeException { 7 usages 7 inheritors 2 woodgeon
    public BusinessException(String message) { 7 usages 2 woodgeon
        super(message);
    }
}
```

- global
 - exception
 - ⚡ BusinessException
 - ⚡ DatabaseException
 - ⚡ DuplicateLeaveRequestException
 - ⚡ InsufficientLeaveException
 - ⚡ InvalidLeaveStatusException
 - ⚡ LeaveDeductionException
 - ⚡ LeaveRequestNotFoundException
 - ⚡ UnauthorizedAccessException

사용자 정의 예외를 통한 비즈니스 규약 정의

예외 커스텀을 통한 비즈니스 규약을 정의했습니다. 이에따라 단순 에러 처리를 넘어 규칙과 의미를 명확히 했습니다.

global/exception/BusinessException

UnauthorizedAccessException : 휴가 결재 및 인사평가는 권한이 있는 관리자나 같은 부서 내에서만 허용한다.

InsufficientLeaveException : 부여된 잔여 연차 범위를 초과하는 휴가 신청은 허가하지 않는다.

DuplicateLeaveRequestException : 한 사원이 동일한 기간에 중복하여 휴가를 신청하거나 승인받을 수 없다.

InvalidLeaveStatusException : 이미 결재 완료(승인/반려)되거나 취소된 신청 건은 재작업할 수 없다.

의미 규정이 중요한 이유

- 모호한 예외의 한계

코드, 혹은 로그 상으로 그것이 어떤 규약을 위반했는지 알기 어려움

- 예외 커스텀을 통한 의미 명확화

예외 클래스 이름 자체가 곧 회사의 업무 규칙이므로 별도의 문서 확인 없이 비즈니스를 설명함,

예외 처리는 단순 기술적인 방어 코드가 아닌 비즈니스 도메인의 규칙을 명시하는 행위

휴가 신청은 중복 신청과 잔여 연차를 먼저 검증

정책 객체가 반환한 객체 내부 로직에 의해 필요한 연차 값인 `requiredAnnualLeave`를 계산하고 이것을 현재 `remainingAnnualLeave`를 비교합니다.



입구 컷

무의미한 DB 커넥션 생성과 트랜잭션 구동 낭비를 막기 위해 검증을 서비스 상단에서 처리합니다.

관리자 승인 처리는 상태 변경, 근태 반영, 연차 차감을 함께 처리

결재 상태 변경과 연차 차감은 동시에 성공하거나 동시에 실패해야 하는 All or Nothing 구조입니다.



흐름 제어

중간에 쿼리가 하나라도 실패하면 Exception을 throw하여 rollback이 확실하게 동작하도록 했습니다.

연차 생성, 잔여 계산, 만료/갱신은 Trigger·Procedure·Scheduler로 처리

```
create or replace PROCEDURE PROC_RENEW_ANNUAL_LEAVE AS
BEGIN
  -- 1. 만료된 연차 처리
  UPDATE ANNUAL_LEAVE
  SET IS_ACTIVE = 'N'
  WHERE EXPIRED_AT < TRUNC(SYSDATE)
  AND IS_ACTIVE = 'Y';

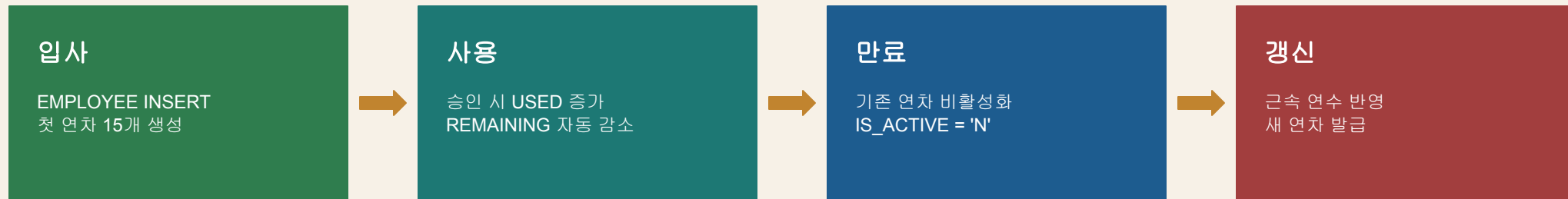
  -- 2. 새 연차 발급 (남은 연차 계산 로직 제거)
  FOR emp_rec IN (
    SELECT
      e.EMP_ID,
      e.HIRE_DATE,
      (EXTRACT(YEAR FROM SYSDATE) - EXTRACT(YEAR FROM e.HIRE_DATE)) as work_years
    FROM EMPLOYEE e
    JOIN ANNUAL_LEAVE al ON e.EMP_ID = al.EMP_ID
    WHERE al.IS_ACTIVE = 'N'
    AND al.EXPIRED_AT = TRUNC(SYSDATE) - 1
  ) LOOP
    INSERT INTO ANNUAL_LEAVE (
      ANNUAL_LEAVE_ID,
      EMP_ID,
      GRANTED_AT,
      EXPIRED_AT,
      GRANTED_ANNUAL_LEAVE,
      USED_ANNUAL_LEAVE,
      -- REMAINING_ANNUAL_LEAVE 컬럼 제외 (트리거가 자동 처리)
      IS_ACTIVE
    ) VALUES (
      SEQ_ANNUAL_LEAVE_ID.NEXTVAL,
      emp_rec.EMP_ID,
      TRUNC(SYSDATE),
      ADD_MONTHS(TRUNC(SYSDATE), 12) - 1,
      15 + emp_rec.work_years,
      0, -- USED만 0으로 넣어주면 됨
      'Y'
    );
  END LOOP;
ENDURE PROC_RENEW_ANNUAL_LEAVE > BEGIN > FOR emp_rec
```

```
create or replace TRIGGER TRG_CALC_REMAINING_LEAVE
BEFORE INSERT OR UPDATE ON ANNUAL_LEAVE
FOR EACH ROW
BEGIN
  -- 잔여 = 부여 - 사용 로직을 DB 단에서 강제
  :NEW.REMAINING_ANNUAL_LEAVE := :NEW.GRANTED_ANNUAL_LEAVE - :NEW.USED_ANNUAL_LEAVE;
END;
```

이름	값
1 OWNER	HDF
2 JOB_NAME	JOB_ANNUAL_LEAVE_DAILY_BATCH
3 JOB_CLASS	DEFAULT_JOB_CLASS
4 COMMENTS	(null)
5 ENABLED	TRUE
6 CREDENTIAL_NAME	(null)
7 DESTINATION	(null)
8 PROGRAM_NAME	(null)
9 JOB_TYPE	STORED_PROCEDURE
10 JOB_ACTION	PROC_RENEW_ANNUAL_LEAVE
11 NUMBER_OF_ARGUMENTS	0
12 SCHEDULE_OWNER	(null)
13 SCHEDULE_NAME	(null)
14 SCHEDULE_TYPE	CALENDAR
15 START_DATE	26/05/15 00:00:00.000000000 ASIA/SEOUL
16 REPEAT_INTERVAL	FREQ=DAILY; BYHOUR=0; BYMINUTE=0; BYSECOND=0
17 END_DATE	(null)
18 EVENT_QUEUE_OWNER	(null)
19 EVENT_QUEUE_NAME	(null)
20 EVENT_QUEUE_AGENT	(null)
21 EVENT_CONDITION	(null)
22 FILE_WATCHER_OWNER	(null)
23 FILE_WATCHER_NAME	(null)
24 CONNECT_CREDENTIAL_OWNER	(null)
25 CONNECT_CREDENTIAL_NAME	(null)
26 STORE_OUTPUT	TRUE

연차 생성, 잔여 계산, 만료/갱신은 Trigger·Procedure·Scheduler로 처리

Java 애플리케이션은 항상 `IS_ACTIVE = 'Y'`인 데이터만 조회하여 최신 잔액을 확인합니다.



Scheduler

매일 자정에 프로시저를 스케줄러로 실행합니다. 사용자마다 부여일, 마감일이 다르기 때문에 마감일 계산 로직을 스케줄러로 처리했고 마감일이 현재 날짜와 같으면 신규 연차를 부여합니다.

급여

주요 기능

- 급여 생성
- 급여 목록/상세 조회
- 급여 수정/삭제
- 급여 상태 변경
- 기타수당 관리
- 기본급, 성과급 관리

- **급여 생성 시 스케줄러, 프로시저 적용**

매월 1일 스케줄러가 지난 달 급여 생성 프로시저 실행

- **급여 명세 구조**

PAYROLL, EARNING, DEDUCTION 으로 분리

- **데이터 정합성 보장**

트랜잭션, 트리거로 급여 항목 정합성 보장

- **지급/공제 항목 계산 관리**

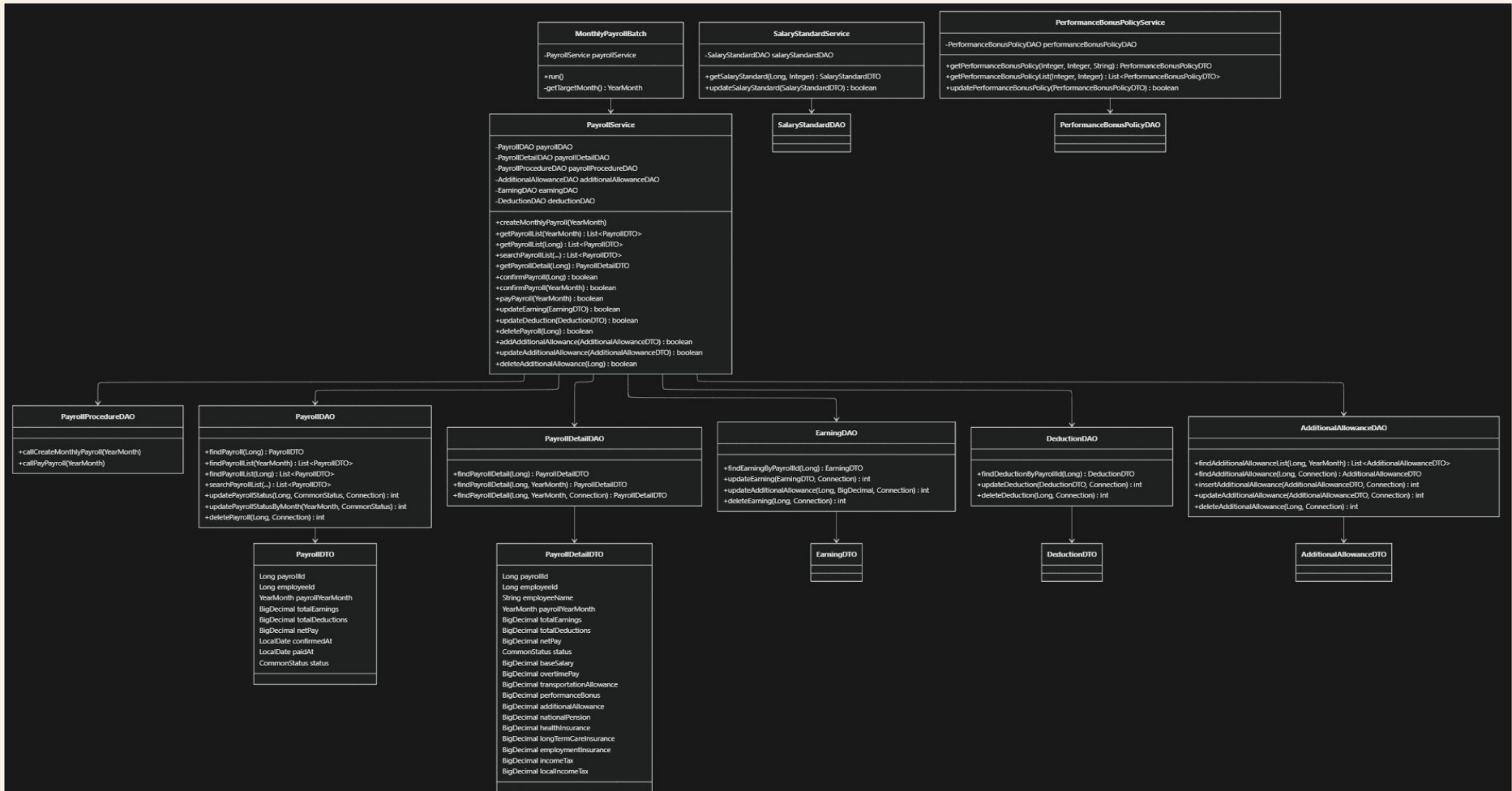
확장성과 유지보수성을 고려해 프로시저에서 공식으로 관리

- **급여 상태 관리**

CALCULATED -> CONFIRM -> PAID 상태 전이 고정

DB 제약조건 및 트리거로 임의 조작 방어

급여 기능 관련 클래스 다이어그램



급여 파트 주요 기능

직원들의 기본급, 수당, 공제액을 기준으로 실지급 급여를 계산하고 조회/등록/수정/삭제할 수 있도록 급여 관리 기능을 구현

1. 급여 생성

1. 매월 1일 02:00 스케줄러 설정
2. 월별 급여 생성 프로시저 호출
(CREATE_MONTHLY_PAYROLL)
3. 급여 명세 생성을 위한 세부 프로시저 호출
(근태 마감 확인, 급여 명세 생성, 지급 및 공제 항목 계산 등)

2. 목록/상세 조회

- 월별 급여 목록 조회
- 직원 개인 급여 목록 조회
- 월, 상태, 부서, 직급, 사번 조건 검색

4. 급여 수정/삭제

- 지급, 공제 각각의 항목 수정/삭제 가능
- 각 항목 수정 시 급여 명세 자동 업데이트 (트랜잭션, 트리거)

3. 급여 상태 변경

- CALCULATED -> CONFIRMED -> PAID 상태 전이 고정
- 인사 담당자가 급여 명세 개별, 전체 확정(CONFIRM) 가능
- 매월 10일 스케줄러로 지급 처리 배치

5. 기타 수당 관리

- 각 직원은 여러 건의 기타 수당을 가질 수 있음
- 인사 담당자가 해당 직원의 기타수당 항목 생성/수정/삭제 가능
- 기타 수당 생성/수정/삭제 시 급여 명세에도 자동 반영

6. 기본급/성과급 정책

- 기본급 정책은 직급/호봉 기준으로 관리
- 성과급 정책은 평가연도/분기/등급 별로 관리
- 인사 담당자가 정책수정 권한을 가짐

설계 1: 급여 생성 로직

애플리케이션에서 처리 vs DB에서 처리

도메인 특성

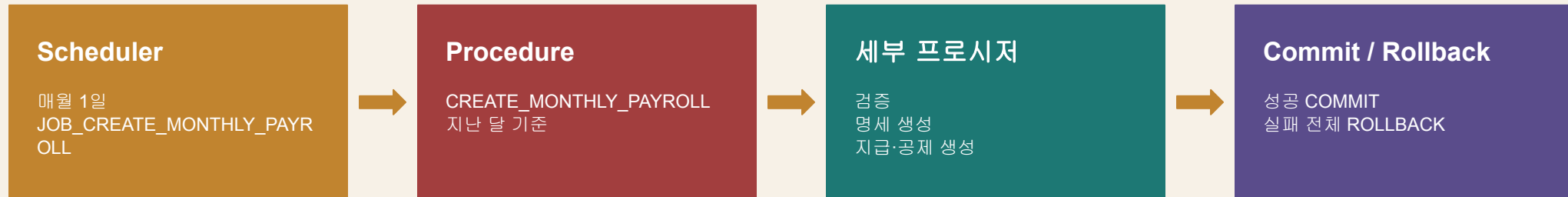
인사 담당자(사용자)가 급여 생성 과정에 관여하지 않기 때문에 DB에서 일괄 처리해도 상관 없음

성능 고려

애플리케이션 단에서 급여 계산 시 각 지원의 정보와 급여 정보를 DB에서 일일이 통신하는 것은 성능이 좋지 않을 것이라 판단

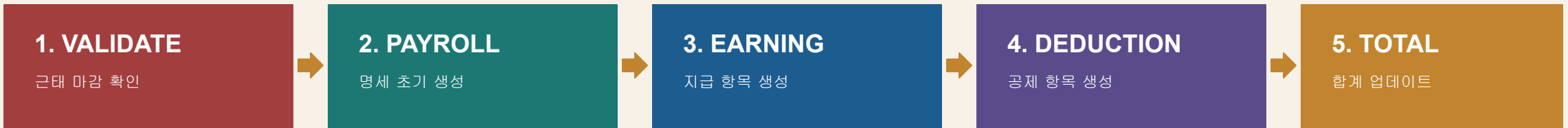


직원을 일일이 조회 및 통신하면서 애플리케이션 단에서 급여를 생성하기 보다,
프로시저로 DB에서 급여 일괄 생성 및 저장하도록 설계



설계 1: 급여 생성 로직

CREATE_MONTHLY_PAYROLL 프로시저 구성



```
1 create or replace PROCEDURE CREATE_MONTHLY_PAYROLL (  
2     p_payroll_year_month IN DATE  
3 )  
4 IS  
5 BEGIN  
6     IF p_payroll_year_month IS NULL THEN  
7         RAISE_APPLICATION_ERROR(-20000, '급여 생성 월은 필수입니다.');8     END IF;  
9  
10    VALIDATE_MONTHLY_PAYROLL(p_payroll_year_month);  
11    INSERT_MONTHLY_PAYROLL(p_payroll_year_month);  
12    INSERT_MONTHLY_EARNING(p_payroll_year_month);  
13    INSERT_MONTHLY_DEDUCTION(p_payroll_year_month);  
14    UPDATE_MONTHLY_PAYROLL_TOTAL(p_payroll_year_month);  
15  
16    COMMIT;  
17 EXCEPTION  
18     WHEN OTHERS THEN  
19         ROLLBACK;  
20         RAISE;  
21 END;  
22
```

실패 처리

생성 실패 시 전체 rollback, 이후 인사관리자가 수동 배치를 실행할 수 있도록 설계

설계 2: 급여 명세 구조

급여 명세를 지급, 공제 내역으로 분리해서 관리

- 급여 명세 항목을 하나의 PAYROLL 테이블에 넣기에는 지급 항목과 공제 항목의 성격이 명확히 다름
- 지급, 공제는 각각 계산 로직을 담고 있어 덩치가 큼



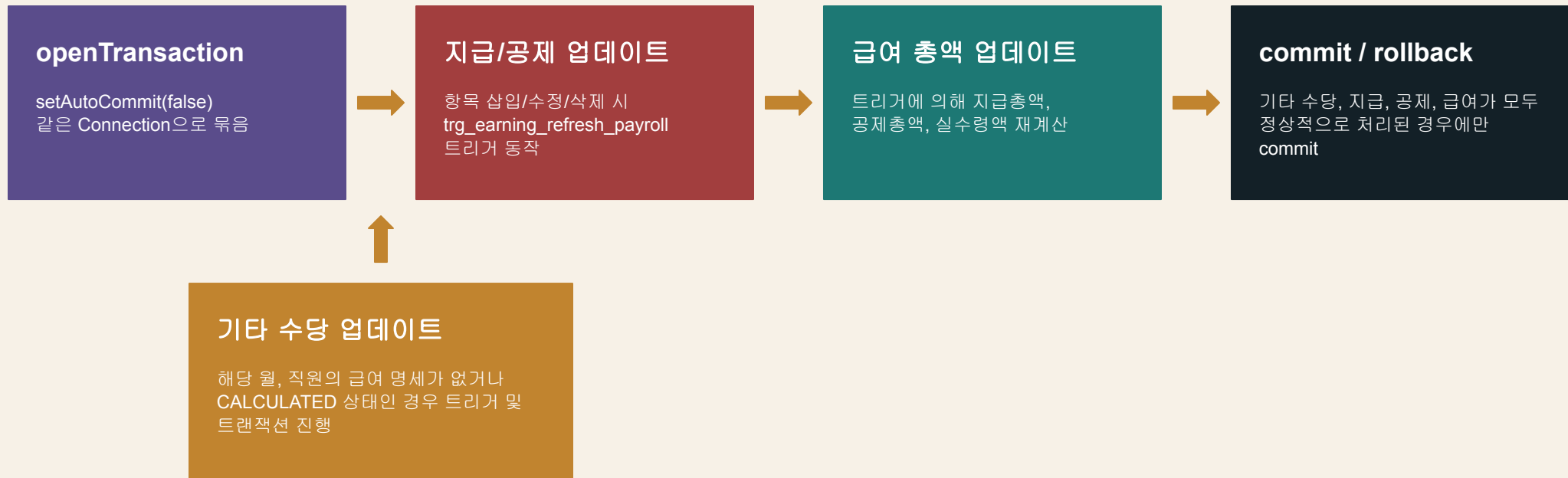
설계 3: 데이터 정합성 문제 해결 -> Transaction, Trigger

급여는 회계성 데이터이기 때문에 부분 성공을 허용하지 않음 -> 원자적 처리 필요

ex.1) 급여 명세는 생성됐는데 공제 내역 생성이 실패하는 경우

ex.2) 지급 항목을 수정했는데 급여 명세의 합계에는 반영되지 않은 경우

처리(설계) 방식



설계 3: 데이터 정합성 문제 해결 -> Transaction, Trigger

ex) 급여 명세 생성 이후, 지급 항목을 수정하는 경우

구현 내용 1. 같은 Connection을 넘겨 하나의 트랜잭션으로 설정

PayrollService.java / updatePayrollDetail(EarningDTO, DeductionDTO, Connection)

```
int earningRowcount = earningDAO.updateEarning(earning, conn);
// trg_earning_refresh_payroll 실행

int deductionRowcount = deductionDAO.updateDeduction(deduction, conn);
// trg_deduction_refresh_payroll 실행

if (earningRowcount != 1 || deductionRowcount != 1) {
    rollback(conn);
    return false;
}
conn.commit();
```

구현 내용 2. 트리거 설정

```
create or replace trigger trg_earning_refresh_payroll
after insert or update or delete on earning
for each row
declare
    v_payroll_id number;
    v_total_earnings number(15,2);
    v_total_deductions number(15,2);
begin
    v_payroll_id := nvl(:new.payroll_id, :old.payroll_id);

    if deleting then
        v_total_earnings := 0;
    else
        v_total_earnings :=
            nvl(:new.base_salary, 0)
            + nvl(:new.overtime_pay, 0)
            + nvl(:new.transportation_allowance, 0)
            + nvl(:new.performance_bonus, 0)
            + nvl(:new.additional_allowance, 0);
    end if;

    select nvl(sum(
        nvl(national_pension, 0)
        + nvl(health_insurance, 0)
        + nvl(long_term_care_insurance, 0)
        + nvl(employment_insurance, 0)
        + nvl(income_tax, 0)
        + nvl(local_income_tax, 0)
    ), 0)
    into v_total_deductions
    from deduction
    where payroll_id = v_payroll_id;

    update payroll
    set total_earnings = v_total_earnings,
        total_deductions = v_total_deductions,
        net_pay = v_total_earnings - v_total_deductions
    where payroll_id = v_payroll_id;
end;
```

설계 4: 지급/공제 항목 계산 관리

급여 계산 로직 및 가중치를 어떻게 관리할 것인가?

실제로 급여 계산은 사내 규칙과 법이 복잡하기 때문에
1차 프로젝트에 공식을 모두 도입하기에는 한계가 있었음
-> 급여 계산 로직 일정 부분 단순화한 상태

따라서 앞으로의 확장성 및 유지보수성을 고려해서 다음과 같이 설계함.

1. 가중치 테이블 -> 프로시저 코드로 고정

법이나 회사 내규가 바뀌어야 하는 규모의 작업이고, 인사 담당자가
임의로 변경하는 일은 드물다고 판단

2. 항목 계산 -> 식으로 관리

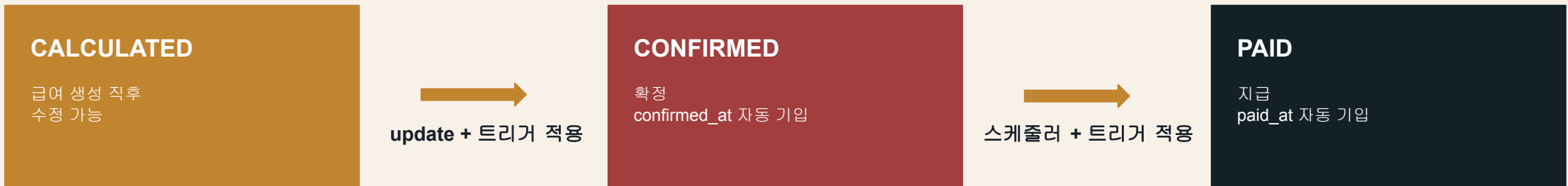
현재는 기준 액수 * 가중치로 되어 있어 일괄 고정할 수 있었지만,
확장성을 고려하여 항목 계산은 각각 식으로 관리

ex) INSERT_MONTHLY_DEDUCTION (PRC)

```
deduction_amount AS (
  SELECT
    et.payroll_id,
    ROUND(et.total_earnings * 0.045) AS national_pension,
    ROUND(et.total_earnings * 0.03545) AS health_insurance,
    ROUND(ROUND(et.total_earnings * 0.03545) * 0.1295) AS long_term_care_insurance,
    ROUND(et.total_earnings * 0.009) AS employment_insurance,
    ROUND(
      CASE
        WHEN et.total_earnings <= 3000000 THEN
          et.total_earnings * 0.03
        WHEN et.total_earnings <= 5000000 THEN
          3000000 * 0.03
          + (et.total_earnings - 3000000) * 0.06
        ELSE
          3000000 * 0.03
          + 2000000 * 0.06
          + (et.total_earnings - 5000000) * 0.10
      END
    ) AS income_tax
  FROM earning_total et
),
deduction_total AS (
  SELECT
    da.*,
    ROUND(da.income_tax * 0.10) AS local_income_tax
  FROM deduction_amount da
)
```

설계 5: 급여 상태 CALCULATED → CONFIRMED → PAID로 고정

CommonStatus Java enum, DB 제약 조건, 상태 변경 트리거로 관리



```
create or replace trigger trg_payroll_status
before update of status on payroll
for each row
begin
    if :old.status = 'CALCULATED' and :new.status = 'CONFIRMED' then
        :new.confirmed_at := sysdate;

    elsif :old.status = 'CONFIRMED' and :new.status = 'PAID' then
        :new.paid_at := sysdate;

    else
        raise_application_error(-20001, 'CALCULATED -> CONFIRMED -> PAID만 허용');
    end if;
end;
```

방어

제대로 된 전이가 아닌 경우
raise_application_error로 rollback

설계 5: CONFIRMED_AT, PAID_AT은 상태 변경 트리거가 기록

CONFIRM 이후 급여 수정은 트리거로 막습니다.

trg_payroll_status

```
BEFORE UPDATE OF status ON payroll
FOR EACH ROW
BEGIN
  IF :old.status = 'CALCULATED'
    AND :new.status = 'CONFIRMED' THEN
    :new.confirmed_at := SYSDATE;

  ELSIF :old.status = 'CONFIRMED'
    AND :new.status = 'PAID' THEN
    :new.paid_at := SYSDATE;

  ELSE
    raise_application_error(
      -20001,
      'CALCULATED -> CONFIRMED -> PAID만 허용'
    );
  END IF;
END;
```

처리 기준

- **확정**
CALCULATED → CONFIRMED
- **지급**
CONFIRMED → PAID
- **잠금**
trg_earning_lock, trg_deduction_lock,
trg_additional_allowance_lock

협업

<> 커밋 컨벤션

협업 효율을 위한 일관된 커밋 메시지 규칙을 준수합니다.

- **feat:** 새로운 기능 추가
- **fix:** 버그 수정
- **docs:** 문서 수정 (README 등)
- **style:** 코드 포매팅, 세미콜론 등
- **refactor:** 코드 리팩토링
- **test:** 테스트 코드 작업
- **chore:** 기타 설정, 패키지 관리 등

📁 브랜치 전략

main: 최종 배포용 안정 릴리스

develop: 중심 개발 브랜치

브랜치 네이밍 컨벤션

작업 성격(feat, fix 등)을 타입으로 사용

feat/#23

fix/#45

↔ 개발 워크플로우

- 1 기능 개발: develop에서 feature 파생
- 2 작업 및 커밋: 컨벤션에 맞춰 진행
- 3 리뷰 및 병합: PR 생성 및 코드 리뷰
- 4 테스트: 병합 후 통합 테스트 수행
- 5 최종 배포: main 브랜치 머지

프로젝트 회고 및 리팩토링 사항

1. 예외 처리 고도화

사용자 정의 예외 처리 커스텀이
완벽하지 않음 (현재 제한적으로
적용)

2. 근태 로직 추상화

근태 관리 부분에 인터페이스를
도입하여 객체지향적 추상화 및
유연한 구조로 개선 가능

3. 코드 컨벤션 정립

코드 스니펫 및 타입 정의서
미작성으로 인해 개발자별
변수명/메서드명 불일치 문제 발생

— Thank you

Q&A